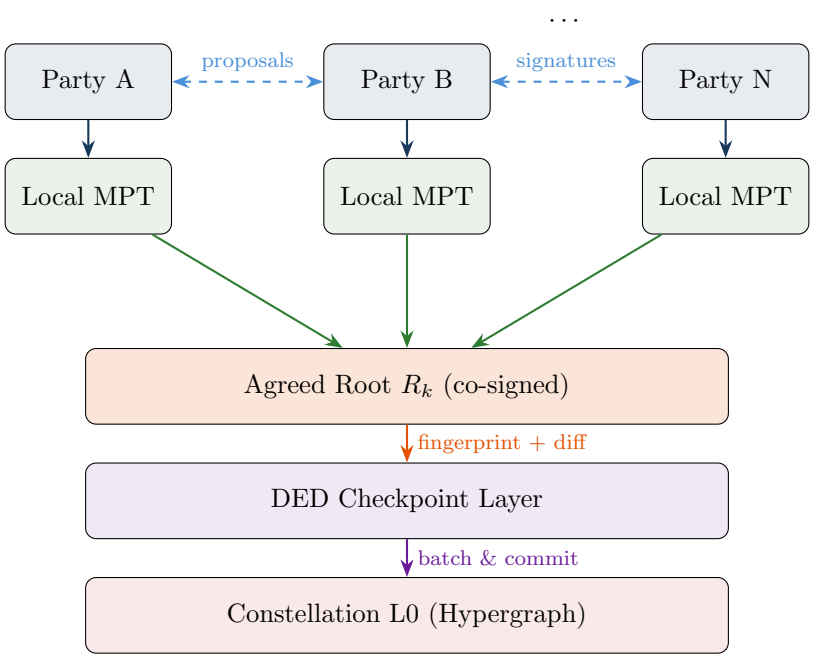
DED Multi-Party State Protocol

Verifiable Multi-Party Computation via Committed State Tries

Technical Proposal — v0.1

February 2026



Status:	Draft
Authors:	Constellation Network Engineering
Extends:	DED Agent WorldState v0.1
Package:	`@constellation-network/ded-agent-worldstate`
Dependencies:	`@constellation-network/ded-sdk`, `@constellation-network/metagraph-sdk`

Contents

1 Executive Summary **3**

2 Motivation **3**
2.1 The Multi-Party Trust Gap . . . 3
2.2 Why DED as the Foundation . . . 4

3 Relationship to v1 **4**

4 System Architecture **4**
4.1 Two-Layer State Model . . . 4
4.2 Architecture Diagram . . . 6

5 Protocol Specification **6**
5.1 Protocol Spec . . . 6
5.2 Schema-Defined State Paths . . . 7
5.2.1 Path Encoding . . . 8
5.3 Transition Functions . . . 8
5.4 Protocol Genesis . . . 9
5.5 State Evolution . . . 9
5.6 Quorum Model . . . 10

6 Communication Layer **11**

7 Data Availability **12**
7.1 State Diff Model . . . 12
7.2 Upload Strategy . . . 12
7.3 Cost Analysis . . . 12

8 Security Model **13**
8.1 Signed Transition Envelope . . . 13
8.2 Threat Model . . . 13
8.2.1 Equivocation Detection . . . 13
8.2.2 Witness Pre-Commitment . . . 14
8.2.3 ECDSA Signature Considerations . . . 14

9 Dispute Resolution **14**
9.1 Phase 1: Client-Side Dispute Resolution . . . 14
9.2 Phase 2: Arbiter Service . . . 15

10 Sigma Protocols and Zero-Knowledge Proofs **15**
10.1 The Vision . . . 15
10.2 Sigma Protocol Primer . . . 15
10.3 Composition with MPT Proofs . . . 15
10.4 Feasibility Assessment . . . 16
10.5 Recommended Path . . . 16

11 API Surface **17**

12 Protocol Upgrades **18**

13 Implementation Roadmap **19**

14 Comparison with Related Systems **19**

Appendix A: Cryptographic Commitment Properties **20**

Appendix B: DED Fingerprint Encoding **20**

Appendix C: Open Questions **21**

1 Executive Summary

This proposal extends the DED Agent WorldState (v1) from single-party state commitment to a **Multi-Party Verifiable State Protocol (MPVSP)** where N parties maintain a shared Merkle Patricia Trie, agree on state transitions via co-signature, and checkpoint committed roots to the DED notarization platform.

The core insight is that DED serves as a **periodic checkpoint layer** — not a real-time consensus engine. Parties advance state off-chain at communication speed (sub-second), while DED provides the cryptographic anchoring that makes historical state provable and disputes resolvable. This two-layer architecture mirrors the relationship between Layer 2 rollups and Layer 1 settlement, but without requiring smart contracts or on-chain execution.

Key design decisions:

- **Zero server changes**: The entire protocol lives in the client SDK. DED's existing fingerprint and upload APIs are used unmodified.
- **Transport-agnostic**: The SDK defines a `Transport` interface; parties bring their own communication channel (WebSocket, HTTP, MQTT, etc.).
- **Git-referenced transition functions**: Allowed state transitions are defined by code in a versioned repository. The protocol spec references a git commit hash; the SDK loads and executes validators from that code.
- **State diffs via `/upload`**: Each committed transition includes the state diff uploaded as a document, enabling any party to reconstruct state from the DED-committed history.
- **Phased delivery**: Phase 1 targets bilateral (N=2) protocols. Phase 2 extends to small consortia (N=3–10). Phase 3 introduces zero-knowledge property proofs and WASM transition functions.

2 Motivation

The v1 WorldState proposal established that a single autonomous agent can maintain a verifiable internal state by committing MPT roots to DED. This solved the *self-verification* problem: an agent proving the integrity of its own state history.

However, real-world workflows rarely involve a single party acting alone. Supply chain handoffs require shipper and receiver agreement. Escrow transactions need buyer, seller, and arbiter coordination. Regulatory compliance demands multi-party attestation. In each case, the critical question extends from "what did *I* know?" to "what did *we* agree?"

2.1 The Multi-Party Trust Gap

When multiple parties must agree on shared state, three new challenges emerge beyond single-party commitment:

1. **Agreement** — How do parties reach consensus on state transitions without a central authority? Who proposes changes, and how are conflicts resolved?
2. **Accountability** — If a party violates the agreed protocol, how is this detected and proven to third parties? Can we produce cryptographic evidence of misbehavior?
3. **Verifiability** — Can an external auditor verify the multi-party interaction history without trusting any individual party? Can parties prove properties about their state without revealing the state itself?

2.2 Why DED as the Foundation

DED provides three properties that make it uniquely suited as the anchoring layer for multi-party protocols:

- **Neutral commitment log**: DED is operated by a network (Constellation Hypergraph), not a single party. Commitments are tamper-evident and publicly verifiable.
- **Document storage**: The `POST /v1/fingerprints/upload` route provides hash-verified document storage alongside fingerprint commitments, enabling state diff persistence.
- **Existing proof infrastructure**: DED already generates MPT inclusion proofs (`GET /v1/fingerprints/{hash}/proof`), providing the cryptographic foundation for dispute resolution.

3 Relationship to v1

This proposal is a strict extension of the v1 Agent WorldState. All v1 concepts — per-stream MPTs, chain linking via `documentId`, checkpoint and recovery — remain unchanged. Multi-party state is built *on top of* single-party streams:

Architectural Principle

A multi-party protocol instance is an orchestration of single-party streams. Each party writes to their own stream. Shared state is written by the leader after quorum is reached. The protocol layer coordinates which streams are written and when.

Concretely, a protocol instance with members `[A, B, C]` maps to the following v1 streams:

Logical Path	v1 Stream	Writer
`/protocol/*`	`stream(proto, ``protocol'', specHash)`	Genesis only
`/state/A/*`	`stream(proto, ``A'', specHash)`	Party A
`/state/B/*`	`stream(proto, ``B'', specHash)`	Party B
`/state/C/*`	`stream(proto, ``C'', specHash)`	Party C
`/shared/*`	`stream(proto, ``shared'', specHash)`	Leader (after quorum)
`/audit/*`	`stream(proto, ``audit'', specHash)`	Leader (after quorum)

The protocol-level MPT is a **unified trie** that spans all these logical paths. Each party maintains a complete local copy of this unified trie. The v1 per-stream MPTs provide the DED-committed chain for each logical partition; the protocol-level MPT is the single root that binds them all.

4 System Architecture

4.1 Two-Layer State Model

The protocol operates across two distinct state layers with different consistency and finality guarantees:

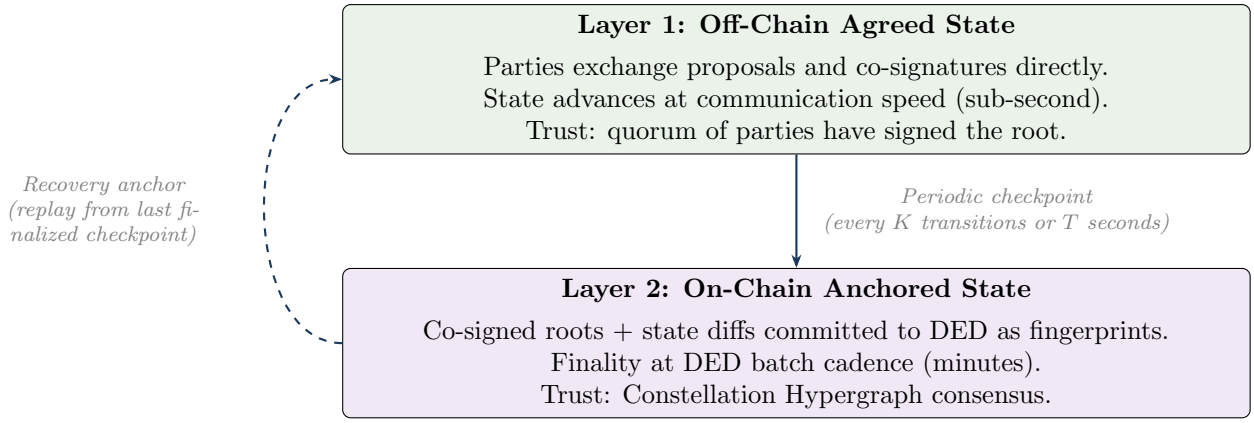


Figure 1: Two-layer state model. Layer 1 is fast but requires party availability. Layer 2 is slow but provides durable, publicly verifiable anchoring.

The Finality Gap

Between off-chain agreement and DED confirmation, there is a window of 1–5 minutes where the agreed state is not yet anchored. During this window, equivocation is theoretically possible. The protocol mitigates this via **epoch numbering** and **previousRoot chaining** (Section 8).

4.2 Architecture Diagram

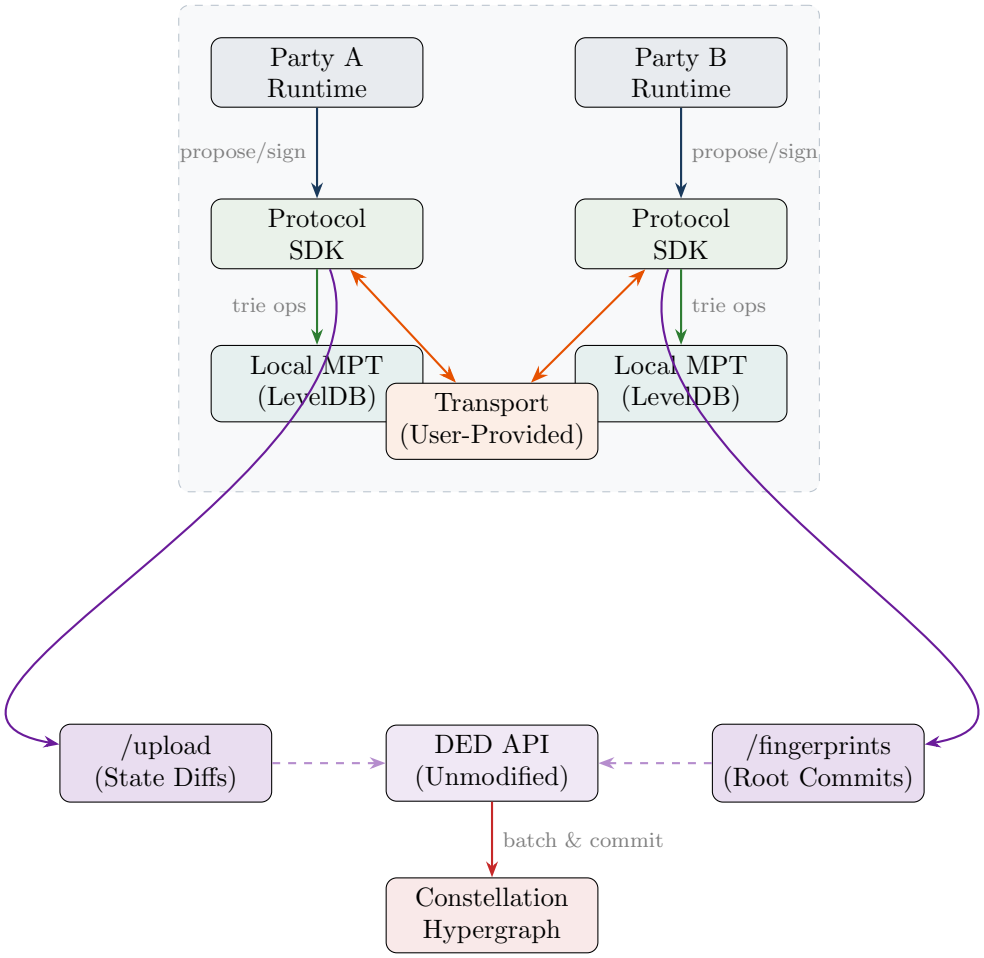


Figure 2: Multi-party architecture. The entire protocol runs client-side. DED serves as an unmodified checkpoint and storage layer.

5 Protocol Specification

5.1 Protocol Spec

A `ProtocolSpec` defines the rules, participants, and allowed transitions for a multi-party interaction. The spec is a structured object whose SHA-256 hash serves as its identity:

```
interface ProtocolSpec {
  name: string;                          // Human-readable name
  version: number;                       // Spec version (monotonic)
  codeRef: {
    type: 'git';
    repository: string;                  // Git repository URL
    commit: string;                      // Full SHA-1 commit hash
    entrypoint: string;                  // Path to transition module
  };
  members: Record<string, MemberSpec>;   // memberId -> spec
  transitions: Record<string, TransitionSchema>;
  quorum: QuorumRule;
  config: ProtocolConfig;
}
```

```typescript
interface MemberSpec {
  publicKey: string;                      // SECP256K1 public key (hex)
  roles: string[];                        // Authorized transition types
}

interface TransitionSchema {
  proposer: string | string[];            // Who may propose
  requiredSigners: string[] | QuorumRule; // Who must co-sign
  statePaths: {
    reads: string[];                      // Paths read during validation
    writes: string[];                     // Paths written on commit
  };
}

type QuorumRule =
  | { type: 'all' }                       // All members
  | { type: 'threshold'; min: number }    // At least min members
  | { type: 'roles'; require: string[] } // Specific members by ID

interface ProtocolConfig {
  checkpointInterval: number;             // Transitions between DED commits
  proposalTimeoutMs: number;              // ms before proposal expires
  leaderElection: 'round-robin' | 'proposer-defined';
  disputeWindowMs: number;                // ms for dispute submission
}
```

Spec Hash

specHash = SHA-256(RFC8785(ProtocolSpec))

The spec hash is computed over the canonical JSON serialization of the `ProtocolSpec` object. All parties must compute the same hash to participate. The spec hash appears as the `appId` in DED fingerprint submissions, binding each commitment to its governing protocol.

5.2 Schema-Defined State Paths

The protocol's unified MPT uses a hierarchical path namespace with reserved prefixes:

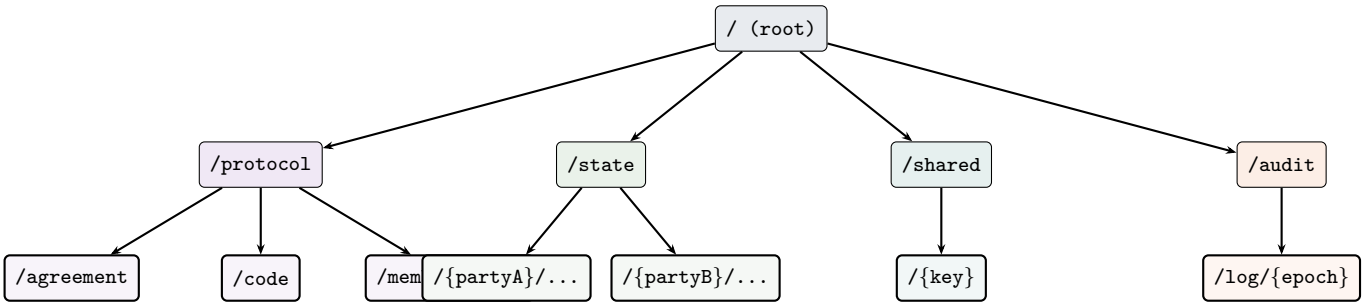


Figure 3: Hierarchical path namespace in the unified protocol MPT.

Prefix	Writer	Mutability	Contents
`/protocol/agreement`	Genesis	Immutable	Hash of legal/commercial terms
`/protocol/code`	Genesis	Immutable*	Git reference (repo + commit)
`/protocol/members/{id}`	Genesis*	Append-only	Public key, roles, status
`/protocol/config`	Genesis	Immutable*	Quorum rules, timeouts, etc.
`/state/{partyId}/...`	Party owner	Mutable	Party-local application state
`/shared/...`	Leader	Mutable	Jointly-owned state (requires quorum)
`/audit/log/{epoch}`	Leader	Append-only	Transition receipts

Table 1: Path namespace registry. *Mutable only via protocol upgrade transitions (Section 12).

5.2.1 Path Encoding

Logical paths are mapped to the 128-hex-char MPT key format required by the DED evidence path structure. The mapping uses deterministic UUID v5 derivation from path components:

$$\text{mptKey} = \text{hex}\Big(\text{UUID5}(\text{instanceId})_{16\text{B}} \| \text{UUID5}(\text{pathSegment}_1)_{16\text{B}} \| \text{UUID5}(\text{pathSegment}_2)_{16\text{B}} \| \text{UUID5}(\text{pathSeg}$$

This preserves compatibility with the server-side `PathEvidenceCalculator` format (`tenantId + streamId + appId + eventId`) while supporting arbitrary logical hierarchies.

5.3 Transition Functions

Transition functions define the allowed state mutations. They are referenced by a git commit hash in the `ProtocolSpec` and loaded as JavaScript/TypeScript modules by the SDK at runtime.

```
// Transition module (referenced by ProtocolSpec.codeRef)
// Repository: github.com/acme/supply-chain-protocol
// Entrypoint: src/transitions/index.ts

import { z } from 'zod';
import type { TransitionModule, StateReader } from
  '@constellation-network/ded-agent-worldstate/protocol';

export const shipment_dispatched: TransitionModule = {
  schema: z.object({
    trackingId: z.string().min(1),
    dispatchedAt: z.string().datetime(),
    items: z.array(z.object({
      sku: z.string(),
      quantity: z.number().int().positive(),
    })),
  }),

  validate: async (state: StateReader, data: unknown) => {
    const parsed = shipment_dispatched.schema.parse(data);
    const existing = await state.read(
      `/state/shipper/dispatches/${parsed.trackingId}`
    );
    if (existing !== null) {
      return { valid: false, error: 'Duplicate dispatch' };
    }
    return { valid: true };
  },
```

```
  apply: async (state: StateWriter, data: unknown) => {
    const parsed = shipment_dispatched.schema.parse(data);
    await state.write(
      `/state/shipper/dispatches/${parsed.trackingId}`,
      { ...parsed, status: 'dispatched' }
    );
  },
};
```

Determinism Requirement

Transition functions **must be pure and deterministic**. Given the same pre-state and input, every party must compute the same post-state. The SDK enforces this by:

- Providing a sandboxed `StateReader`/`StateWriter` that intercepts all state access.
- Prohibiting side effects (no network, no filesystem, no randomness).
- Recording all read paths as witnesses for dispute resolution.

Future Direction: WASM Transition Functions

JavaScript transition functions are convenient but have known determinism risks across runtimes (floating-point handling, string collation, etc.). A future version may support WebAssembly (WASM) modules as transition functions, providing:

- Cross-platform deterministic execution
- Language-agnostic (Rust, Go, C compiled to WASM)
- Sandboxed by design (no host access without explicit imports)
- Formally verifiable execution semantics

The `ProtocolSpec.codeRef` would gain a `type: 'wasm'` variant pointing to a WASM binary with a known hash.

5.4 Protocol Genesis

A protocol instance is created through a multi-step genesis ceremony:

1. **Spec Agreement**: All parties obtain the same `ProtocolSpec` (e.g., via a shared repository or direct exchange) and verify its hash matches.
2. **Genesis Proposal**: The initiating party constructs the genesis MPT containing:
   - `/protocol/agreement` ← `H(terms_document)`
   - `/protocol/code` ← `codeRef` from spec
   - `/protocol/members/{id}` ← member specs
   - `/protocol/config` ← protocol configuration
3. **Co-Signature**: All members sign the genesis root. This is the only transition that *always* requires unanimity, regardless of the configured quorum.
4. **DED Commitment**: The genesis root is submitted to DED as the first fingerprint in the protocol's chain, with:
   - `documentId` = `mpvsp:{instanceId}:epoch:0`
   - `documentRef` = genesis MPT root hash
   - `proofs` = all members' signatures
5. **State Diff Upload**: The full genesis state is uploaded via `POST /v1/fingerprints/upload` as the initial state snapshot.

5.5 State Evolution

After genesis, state evolves through a propose–validate–sign–commit cycle:

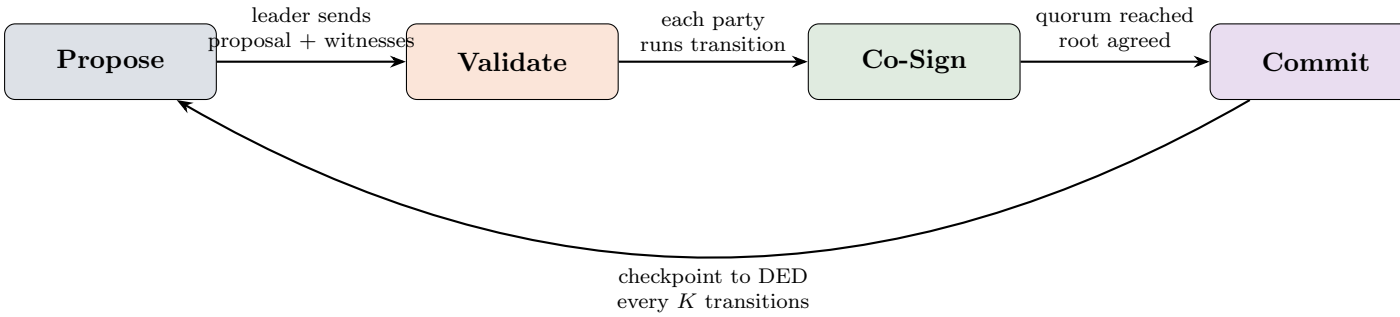


Figure 4: State transition lifecycle. The cycle repeats for each epoch.

Detailed lifecycle for epoch k:

1. **Leader Selection**: For epoch k, the leader is members[k mod N] (round-robin by sorted member ID). If the designated leader does not propose within `proposalTimeoutMs`, the next member becomes leader.
2. **Propose**: The leader constructs a `Proposal`:
   - Transition name and input data (validated against schema)
   - Pre-state root R_{k-1} (the current agreed root)
   - Post-state root R_k (computed by applying the transition)
   - Epoch number k (monotonically increasing)
   - State witnesses: MPT inclusion proofs for all paths read during validation, and the key-value diffs for all paths written
   - Leader's signature over $(R_{k-1}, R_k, k, \text{transitionHash})$
3. **Validate**: Each party independently:
   - Verifies R_{k-1} matches their local root
   - Verifies the state witnesses against R_{k-1}
   - Loads the transition function from the code reference
   - Executes the transition with the witnessed pre-state and input
   - Computes R'_k from the resulting state changes
   - Verifies $R'_k = R_k$ (the proposed root matches their computation)
4. **Co-Sign**: If validation passes, the party signs $(R_{k-1}, R_k, k, \text{transitionHash})$ and sends the signature to the leader via the transport layer.
5. **Commit**: When the leader collects signatures satisfying the quorum rule:
   - The new root R_k becomes the agreed state
   - All parties apply the transition to their local MPT
   - The audit log at `/audit/log/{k}` records the transition metadata
6. **Checkpoint**: Every `checkpointInterval` epochs (or on explicit request), the leader submits the agreed root to DED:
   - `POST /v1/fingerprints` with `documentRef` $= R_k$
   - `POST /v1/fingerprints/upload` with the accumulated state diffs since the last checkpoint

5.6 Quorum Model

The protocol supports configurable quorum rules. For Byzantine fault tolerance, the minimum safe quorum is $2f + 1$ where $f = \lfloor (N-1)/3 \rfloor$:

N	Max Byzantine (f)	Quorum ($2f+1$)	Model
2	0	2	Unanimity (bilateral)
3	0	2	2-of-3
4	1	3	3-of-4
5	1	3	3-of-5
7	2	5	5-of-7
10	3	7	7-of-10

Table 2: BFT quorum sizes by party count.

The `QuorumRule` type allows flexibility:

- {`type: ''all''`} — Unanimity. All members must sign. Required for genesis; optional for transitions.
- {`type: ''threshold'', min: Q`} — At least Q members must sign.
- {`type: ''roles'', require: [''A'', ''B'']`} — Specific named members must sign (e.g., buyer AND seller in an escrow).

6 Communication Layer

The SDK is **transport-agnostic**. It defines a minimal interface that abstracts all inter-party communication:

```
interface Transport {
  /** Send a message to a specific party */
  send(recipientPubKey: string, message: ProtocolMessage): Promise<void>;

  /** Subscribe to incoming messages */
  subscribe(
    callback: (from: string, message: ProtocolMessage) => void
  ): () => void;  // returns unsubscribe function
}

type ProtocolMessage =
  | { type: 'proposal'; proposal: Proposal }
  | { type: 'signature'; epoch: number; signature: string }
  | { type: 'reject'; epoch: number; reason: string }
  | { type: 'sync-request'; fromEpoch: number }
  | { type: 'sync-response'; diffs: StateDiff[] };
```

The SDK provides no default transport implementation. Users choose the communication mechanism appropriate for their deployment:

Environment	**Suggested Transport**	**Notes**
Server-to-server	WebSocket or gRPC	Persistent connections for low-latency proposal exchange
Browser-based	WebRTC DataChannel	Peer-to-peer, no server required for data transfer
Asynchronous	HTTP polling or webhooks	For workflows where real-time exchange is not critical
Serverless	Queue (SQS, Pub/Sub)	For Lambda/Cloud Function deployments

Table 3: Suggested transport mechanisms by deployment environment.

7 Data Availability

7.1 State Diff Model

Each committed transition produces a **state diff** — the set of key-value pairs that changed:

```
interface StateDiff {
  epoch: number;
  previousRoot: string;  // Pre-state root hash
  newRoot: string;       // Post-state root hash
  transition: string;    // Transition name
  changes: DiffEntry[];  // Changed paths
  witnesses: MerklePatriciaInclusionProof[];  // Pre-state witnesses
}

interface DiffEntry {
  path: string;          // Logical path (e.g., /state/alice/balance)
  previousValue: string | null;  // Hash of previous value (null if new)
  newValue: string | null;       // Hash of new value (null if deleted)
  rawValue?: unknown;    // Optional: the actual new value (for
      reconstruction)
}
```

7.2 Upload Strategy

At each DED checkpoint (every `checkpointInterval` transitions), the accumulated state diffs are uploaded:

1. Serialize the batch of `StateDiff` objects since the last checkpoint as canonical JSON.
2. Compute `diffBundleHash` = SHA-256 of the serialized bundle.
3. Submit via `POST /v1/fingerprints/upload`:
   - Fingerprint: `documentRef` = current agreed root R_k, `documentId` = `mpvsp:{instanceId}:epoch:{k`
   - Document: the serialized diff bundle, keyed by `diffBundleHash`

This ensures that any party — or a third-party auditor — can reconstruct the full state history from DED alone:

1. Query DED for all fingerprints with `documentId` matching `mpvsp:{instanceId}:epoch:*`
2. Download the associated diff bundles from DED storage
3. Replay diffs in epoch order, starting from the genesis snapshot
4. Verify each intermediate root matches the committed `documentRef`

7.3 Cost Analysis

Operation	Credits	USD	Notes
Fingerprint write (checkpoint)	2	\$0.02	Per DED checkpoint
Diff bundle upload	$\lceil$size/100KB$\rceil$	\$0.01/100KB	State diff storage
Fingerprint read (public)	0	Free	Proof retrieval
Example: 10 transitions per checkpoint, 5KB avg diff per transition			
Checkpoint fingerprint	2	\$0.02	—
Diff bundle (50KB)	1	\$0.01	—
Per checkpoint total	**3**	**\$0.03**	—
Per transition (amortized)	**0.3**	**\$0.003**	—

Table 4: Cost per checkpoint and amortized per-transition cost.

8 Security Model

8.1 Signed Transition Envelope

Every state transition is wrapped in a cryptographic envelope that prevents equivocation, replay, and reordering:

```
interface SignedTransitionEnvelope {
  protocolInstanceId: string;  // Unique protocol instance
  specHash: string;            // SHA-256 of ProtocolSpec
  epoch: number;               // Monotonically increasing (uint64)
  previousRoot: string;        // Chain linkage: root at epoch-1
  newRoot: string;             // Proposed root at epoch
  transitionHash: string;      // SHA-256 of (transitionName, inputData)
  proofs: SignatureProof[];    // Co-signatures from quorum
}
```

The signed payload is:

$$\text{signedData} = \text{RFC8785}\Big(\{\text{instanceId}, \text{specHash}, \text{epoch}, \text{previousRoot}, \text{newRoot}, \text{transitionHash}\}\Big)$$

8.2 Threat Model

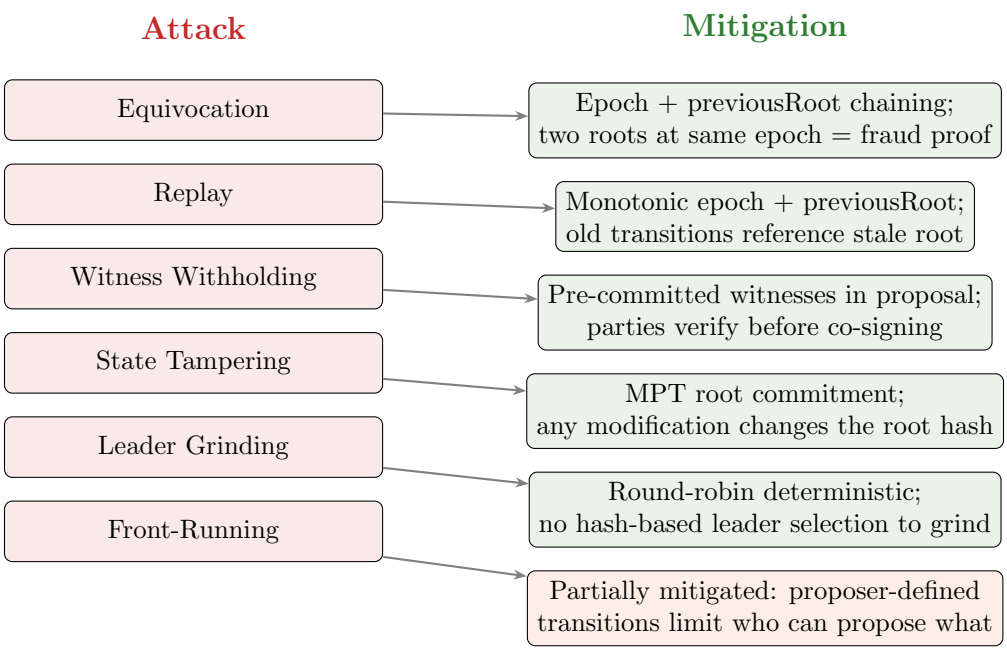


Figure 5: Threat model with mitigations. All critical attacks are addressed. Front-running is partially mitigated via proposer restrictions.

8.2.1 Equivocation Detection

If a leader signs two different roots R_k and R'_k at the same epoch k with the same `previousRoot`, both signatures constitute an **equivocation proof**:

$$\text{EquivocationEvidence} = \big(\text{sig}_1(R_{k-1}, R_k, k),\ \text{sig}_2(R_{k-1}, R'_k, k),\ R_k \neq R'_k\big)$$

Any party holding both signatures can prove the leader acted maliciously. In Phase 1, this triggers social ejection (parties refuse to continue the protocol with the equivocator). In future phases, this could trigger on-chain slashing.

8.2.2 Witness Pre-Commitment

The leader *must* include MPT inclusion proofs for all state paths read during transition validation as part of the proposal. This eliminates the withholding attack entirely:

- Co-signers verify witnesses against R_{k-1} before signing.
- If the leader omits a witness, co-signers reject the proposal.
- All co-signers hold the witnesses at signing time — no party can later claim inability to produce evidence.

8.2.3 ECDSA Signature Considerations

The protocol uses independent ECDSA signatures (one per party) aggregated in the `proofs` array. Two important considerations:

1. **Low-s normalization**: ECDSA signatures are malleable — given (r, s), the signature $(r, n - s)$ is also valid. The SDK enforces $s \leq n/2$ to ensure signature uniqueness, preventing double-counting in quorum validation.
2. **Key registration**: The genesis ceremony requires each member to sign a challenge nonce, proving knowledge of their private key. This prevents rogue key attacks where a malicious party registers a crafted public key.

9 Dispute Resolution

9.1 Phase 1: Client-Side Dispute Resolution

In Phase 1, dispute resolution is implemented entirely in the SDK. The process is "social" — evidence is presented to all parties, and the honest majority acts on it.

1. **Evidence Collection**: The disputing party compiles:
   - The last DED-committed root and epoch (Layer 2 anchor)
   - The disputed proposal with all co-signatures
   - The state witnesses from the proposal
   - Their independent computation of the correct post-state root
2. **Verification**: Any party (or third-party auditor) can verify:
   - Download the diff bundle from DED storage
   - Verify witnesses against the committed pre-state root
   - Re-execute the transition function (from the code reference)
   - Compare the computed post-state root with the disputed root
3. **Resolution**: The honest majority refuses to co-sign further transitions with the misbehaving party. The protocol forks: honest parties continue from the last valid state; the dishonest party is excluded.

```
// Dispute evidence generation
const evidence = await protocol.generateDisputeEvidence({
  disputedEpoch: 42,
  myComputedRoot: 'abc123...',  // What I think the root should be
});
// evidence: { anchor, proposal, witnesses, myComputation, signatures }

// Dispute verification (can run on any party or auditor)
const verdict = await Protocol.verifyDispute(evidence, codeRef);
```

```
// verdict: { valid: true, correctRoot: 'abc123...', faultyParty: 'leader-
    pubkey' }
```

9.2 Phase 2: Arbiter Service

In Phase 2, an independent arbiter service can verify disputes without requiring all parties to be online:

- The arbiter downloads committed state and diffs from DED.
- The arbiter loads and executes the transition function from the code reference.
- The arbiter publishes a signed verdict as a DED fingerprint, creating an on-chain record of the dispute outcome.
- Trust assumption: the arbiter is honest. Multiple independent arbiters can be used for redundancy.

10 Sigma Protocols and Zero-Knowledge Proofs

Research Direction — Not Included in Phases 1–2

This section describes future cryptographic extensions that are **not part of the initial implementation**. They are documented here to establish the research direction and ensure the v1/v2 architecture does not preclude their future addition.

10.1 The Vision

In a multi-party protocol, parties may need to prove properties about their local state without revealing the state itself. For example:

- "My balance at `/state/alice/balance` is ≥ 100" without revealing the exact balance.
- "I hold a credential at `/state/alice/credentials/X`" without revealing the credential details.
- "The sum of all party balances under `/state/*/balance` equals the value at `/shared/total_supply`" (conservation invariant).

10.2 Sigma Protocol Primer

A sigma protocol (Σ-protocol) is a three-move interactive proof between a Prover (P) and Verifier (V):

1. **Commitment**: P sends a random commitment a
2. **Challenge**: V sends a random challenge e
3. **Response**: P sends a response z computed from a, e, and the secret witness

The protocol satisfies *completeness* (honest provers always convince), *special soundness* (extractable witness from two transcripts), and *honest-verifier zero-knowledge* (simulatable without the witness).

Via the **Fiat-Shamir transform**, interactive sigma protocols become non-interactive by replacing the verifier's challenge with $e = \mathrm{H}(a \| \text{statement})$.

10.3 Composition with MPT Proofs

Sigma protocols compose naturally via AND and OR constructions:

- **AND**: Prove knowledge of witnesses w_1 AND w_2 by running both protocols in parallel with the same challenge.

- **OR**: Prove knowledge of w_1 OR w_2 (without revealing which) by simulating one protocol and running the other honestly.

For MPT state proofs, the compound statement would be:

"I know v at path p in trie with root R, and $v \geq$ threshold"

10.4 Feasibility Assessment

Approach	Proof Size	Verify Time	Feasibility
Pedersen + Bulletproofs	∼700 B	∼3 ms	Practical for range proofs on committed values. Requires storing Pedersen commitments alongside SHA-256 hashes in leaves. Does NOT prove MPT inclusion in ZK — the inclusion proof is public.
SNARK (SHA-256 circuit)	∼200 B	∼5 ms	Proves full MPT inclusion + property in ZK. But SHA-256 costs ∼25K R1CS constraints per hash; depth-8 MPT ≈ 200K constraints. Proof generation: seconds to minutes.
SNARK (Poseidon hash)	∼200 B	∼5 ms	Poseidon costs ∼300 constraints per hash. Depth-8 MPT ≈ 2.4K constraints. Practical, but requires replacing the MPT hash function. **Breaking change.**
WASM in ZK (zkWASM)	varies	varies	Prove correct execution of arbitrary WASM in ZK. Experimental; proof generation is very slow. Long-term research direction.

Table 5: Zero-knowledge proof approaches and their feasibility for MPT state proofs.

10.5 Recommended Path

1. **Phase 3a**: Introduce Pedersen commitments as an optional leaf value type. Store $(\text{SHA-256}(v), \text{Pedersen}(v, r))$ in leaves. Enable Bulletproofs range proofs on committed values. The SHA-256 hash preserves backward compatibility; the Pedersen commitment enables ZK property proofs.
2. **Phase 3b**: Evaluate algebraic hash migration (Poseidon) in the Tessellation MPT layer for full in-circuit verification. This is a multi-quarter effort affecting the entire Constellation stack.
3. **Phase 3c**: Explore recursive SNARKs for compressing the entire protocol history into a single proof ("this root R_k is the result of k valid transitions from genesis").

11 API Surface

```
import { WorldState } from '@constellation-network/ded-agent-worldstate';
import { Protocol } from '@constellation-network/ded-agent-worldstate/
    protocol';
import { supplyChainSpec } from '@acme/supply-chain-protocol';

// --- Initialize WorldState (same as v1) ---
const ws = await WorldState.create({
  privateKey: process.env.PRIVATE_KEY!,
  orgId: 'org-uuid',
  baseUrl: 'https://api.ded.example.com',
  apiKey: 'api-key',
  storagePath: './state',
});

// --- Create or join a protocol instance ---
const protocol = await ws.protocol(supplyChainSpec, {
  instanceId: 'handoff-2026-001',
  myRole: 'shipper',
  transport: myWebSocketTransport,  // User-provided
});

// --- Genesis (first party initiates) ---
await protocol.genesis({
  agreementHash: sha256(termsDocument),
});
// Waits for all members to co-sign genesis root

// --- Propose a transition ---
const proposal = await protocol.propose('shipment_dispatched', {
  trackingId: 'TRACK-12345',
  dispatchedAt: new Date().toISOString(),
  items: [{ sku: 'WIDGET-A', quantity: 100 }],
});
// Returns: { proposalId, epoch, previousRoot, newRoot, signatures }

// --- Listen for proposals (other party) ---
protocol.on('proposal', async (proposal) => {
  // SDK auto-validates: schema, witnesses, transition execution, root
  // If valid, proposal.autoValidated === true
  await protocol.sign(proposal.id);
});

// --- Read state ---
const value = await protocol.read('/state/shipper/dispatches/TRACK-12345');
const root = protocol.currentRoot();
const epoch = protocol.currentEpoch();

// --- Generate and verify proofs ---
const proof = await protocol.prove('/state/shipper/dispatches/TRACK-12345');
const valid = await Protocol.verifyProof(proof, protocol.currentRoot());

// --- Protocol lifecycle events ---
protocol.on('committed', ({ epoch, root, fingerprintHash }) => {
  console.log(`Epoch ${epoch} committed to DED: ${fingerprintHash}`);
});

protocol.on('finalized', ({ epoch, globalSnapshotOrdinal }) => {
  console.log(`Epoch ${epoch} finalized on-chain at snapshot ${
      globalSnapshotOrdinal}`);
```

```
});

protocol.on('dispute', ({ epoch, evidence }) => {
  console.log(`Dispute raised at epoch ${epoch}`);
});

// --- Dispute resolution ---
const evidence = await protocol.generateDisputeEvidence({ disputedEpoch: 5
    });
const verdict = await Protocol.verifyDispute(evidence, supplyChainSpec.
    codeRef);

// --- Protocol state inspection ---
const info = await protocol.info();
// { epoch, root, members, lastCheckpointEpoch, lastCheckpointHash }
```

12 Protocol Upgrades

Protocol rules may need to change over time: adding members, modifying transition schemas, or updating the code reference. Upgrades follow a structured process:

1. A member proposes an upgrade by submitting a new `ProtocolSpec` with incremented `version`.
2. The upgrade proposal is a special transition type (`protocol_upgrade`) that *always requires unanimity*, regardless of the configured quorum.
3. On unanimous approval:
   - The current protocol state is checkpointed to DED.
   - The `/protocol/code` and `/protocol/config` paths are updated in the MPT.
   - The new spec hash is recorded in `/audit/log/{epoch}`.
   - Subsequent transitions use the new spec's rules.
4. The old spec hash remains in the audit trail, enabling verification of historical transitions against the rules that were active at the time.

```
// Propose a protocol upgrade
const newSpec = {
  ...currentSpec,
  version: 2,
  members: {
    ...currentSpec.members,
    auditor: { publicKey: auditorPubKey, roles: ['read-only'] },
  },
};

await protocol.proposeUpgrade(newSpec);
// Requires all existing members to co-sign
```

13 Implementation Roadmap

Phase	Deliverable	Description
0	v1 WorldState	Single-party streams, chain linking, checkpoint/recovery, LevelDB persistence, DED integration. *(Prerequisite, per v1 proposal.)*
1	Bilateral Protocols	Two-party protocol instances. Transport interface. Genesis ceremony. Propose/sign lifecycle. State diff upload via `/upload`. Audit log. Client-side dispute evidence generation. Round-trip test suite with in-memory transport.
2	N-Party Protocols	Configurable quorum ($2f + 1$). Round-robin leader election with timeout. N-party signature collection. State sync for late joiners (replay from DED). Protocol upgrades (member add/remove, code update).
3a	Pedersen Commitments	Optional dual-commit leaves (SHA-256, Pedersen). Bulletproofs range proofs on committed values. Does not require server-side changes.
3b	WASM Transitions	WebAssembly transition functions for cross-platform determinism. `codeRef` gains `type: ``wasm''` variant.
3c	Algebraic Hash MPT	Poseidon-based MPT for SNARK-friendly inclusion proofs. Full compound ZK proofs: "value at path p in trie R satisfies predicate P." *Requires Tessellation-level changes.*

Table 6: Implementation roadmap. Each phase builds on the previous.

14 Comparison with Related Systems

	MPVSP on DED	**State Channels**	**Optimistic Rollups**	**zk-Rollups**
State	Arbitrary MPT	Balance pairs	EVM state	EVM state
Parties	N (2–10)	2 (bilateral)	Sequencer + verifiers	Sequencer + prover
Off-chain speed	Sub-second	Instant	N/A (all on-chain)	N/A
Finality	Minutes (DED batch)	Hours (dispute)	∼7 days	Hours (proof gen)
Dispute	Client-side + arbiter	On-chain contract	Fraud proofs	Validity proofs
DA	DED S3 (diffs)	Parties only	L1 calldata	L1 calldata
Trust	Quorum of N	Both parties	1-of-N honest	Cryptographic
Cost/tx	\$0.003 amortized	\$0 off-chain	\$0.01–0.10	\$0.01–0.10
Infra	DED API only	L1 smart contract	L1 + L2 nodes	L1 + prover

Table 7: Comparison with existing multi-party state systems.

MPVSP's unique position: It requires no smart contracts, no L1 gas, no staking, and no specialized infrastructure beyond a DED API key. The tradeoff is weaker dispute enforcement (social/arbiter-based rather than on-chain automated). This makes MPVSP ideal for *cooper-*

ative multi-party workflows (supply chain, compliance, escrow) where parties have real-world relationships, rather than *adversarial trustless* settings.

Appendix A: Cryptographic Commitment Properties

The MPVSP inherits the DED MPT's cryptographic properties:

Binding SHA-256 with type-prefix domain separation (0x00, 0x01, 0x02) ensures that two different sets of (path, value) pairs produce different root hashes with overwhelming probability (2^{-256}). The deterministic insertion order (confirmed by the `MptInsertionOrderDeterm` in the Tessellation test suite) guarantees that the root is uniquely determined by the logical content, not the insertion sequence.

Hiding The MPT commits to *hashes* of values, not plaintext. Leaf nodes store SHA-256(v), not v itself. However, the trie structure leaks metadata: Branch nodes reveal which path prefixes are populated, and Extension nodes reveal shared path segments. Future Pedersen commitment integration (Phase 3a) would add computational hiding for leaf values.

Non-Repudiation
Each party's signature is independently verifiable via SECP256K1. A party that co-signs a root cannot later deny having agreed to that state. The `SignedTransitionEnvelope` binds the signature to a specific (instance, epoch, previousRoot, newRoot) tuple, preventing signature reuse across different protocol contexts.

Temporal Ordering
The epoch counter provides a total order on transitions within a protocol instance. The previousRoot chaining ensures that the order is cryptographically enforced: skipping or reordering epochs would break the hash chain.

Appendix B: DED Fingerprint Encoding

Protocol state commitments are encoded in the existing DED `FingerprintValue` fields without requiring schema changes:

Field	MPVSP Usage	Notes
`orgId`	Organization UUID	Unchanged from v1
`tenantId`	Protocol instance UUID	Derived from instanceId
`eventId`	Random UUID per checkpoint	Unique per fingerprint
`documentId`	`mpvsp:{instanceId}:epoch:{N}`	Protocol chain linking; epoch is monotonic
`documentRef`	Protocol MPT root hash (64 hex)	The agreed state root
`signerId`	Leader's public key	Proposing party
`timestamp`	ISO 8601	Time of checkpoint submission
`version`	1	Schema version
`tags`	`protocol_id`, `spec_hash`, `epoch`, `quorum`	Searchable metadata (max 6 tags)
`proofs`	All co-signer signatures	Quorum evidence

Table 8: Mapping of MPVSP protocol data to DED fingerprint fields.

Appendix C: Open Questions

1. **Transition function determinism**: JavaScript has known non-determinism across runtimes (floating-point, string collation, Map iteration order). Should the SDK enforce a deterministic subset of JS, or mandate WASM from the start? *Current decision: JS with RFC 8785 canonical JSON for all state serialization; WASM as Phase 3b.*

2. **Key rotation**: The genesis ceremony binds member identities to specific SECP256K1 keys. A `key_rotation` transition type should allow a member to replace their key by co-signing with both old and new keys. This is not yet designed.

3. **Partial state visibility**: In some protocols, parties should not see each other's private state paths. The current MPT structure leaks path existence via Branch nodes. Encrypted leaves or separate per-party tries with cross-references may be needed.

4. **Cross-protocol references**: Can one protocol instance reference another's committed state? E.g., "this escrow protocol's delivery confirmation references tracking proof from the logistics protocol." This would require cross-instance MPT proof verification.

5. **FROST threshold signatures**: For Phase 3, migrating from independent ECDSA to FROST (Flexible Round-Optimized Schnorr Threshold) would produce constant-size signatures (64 bytes regardless of N) with native threshold semantics. This requires adding Schnorr support to the Constellation signing stack.

6. **On-chain dispute enforcement**: Phase 1–2 rely on social enforcement. For high-value protocols, on-chain enforcement (metagraph validator extensions to check equivocation proofs and apply penalties) would provide stronger guarantees. This requires changes to the metagraph's `CalculatedState` and `Validator`.